

Módulo 4 SQL

1. Fundamentos de SQL

1.1. Qué es una base de datos

Una **base de datos (BDD)** es un sistema estructurado que permite almacenar, gestionar y recuperar grandes volúmenes de datos de manera eficiente.

Estas son esenciales en cualquier entorno que requiera manejar información, como aplicaciones, sistemas empresariales, sitios web, entre otros.

El principal objetivo de una base de datos es organizar los datos de una forma que permita su acceso rápido y seguro, mientras se garantiza su integridad y coherencia.

Tipos de bases de datos:

- Bases de datos relacionales

Son las más utilizadas y organizan la información en tablas (también conocidas como relaciones).

Estas tablas están formadas por filas (registros) y columnas (atributos o campos).

Las bases de datos relacionales permiten establecer relaciones entre tablas mediante **llaves primarias (Primary Key)** y **llaves foráneas (Foreign Key)**, lo que permite organizar los datos de manera eficiente y realizar consultas complejas.

Por ejemplo, en una base de datos de una tienda, podríamos tener una tabla llamada "Clientes" y otra tabla llamada "Pedidos".

Cada cliente tiene un identificador único (Primary Key) en la tabla "Clientes", y esa misma clave se usa como referencia en la tabla "Pedidos" (Foreign Key), para establecer la relación entre quién realizó cada pedido.

Bases de datos relacionales comunes: **MySQL, PostgreSQL, SQL Server, Oracle.**

- Bases de datos no relacionales

Las bases de datos no relacionales, también conocidas como **NoSQL**, no siguen el esquema de tablas tradicionales y están diseñadas para gestionar grandes volúmenes de datos no estructurados o semiestructurados.

Se usan comúnmente en aplicaciones donde la estructura de datos puede cambiar con frecuencia o cuando se manejan datos masivos que no requieren estricta normalización.

Por ejemplo, en una red social, los datos de los perfiles de usuarios, sus publicaciones, fotos y amigos pueden estar almacenados en una base de datos NoSQL, que permite flexibilidad en el tipo de información que se guarda para cada usuario.

Bases de datos NoSQL comunes: **MongoDB, Cassandra, Redis**

Componentes principales de una base de datos relacional

● **Tablas:** Son el núcleo de una base de datos relacional. Cada tabla contiene datos sobre una entidad específica, como "Clientes" o "Productos".

- **Llave primaria (Primary Key):** Es un campo o combinación de campos que identifican de manera única a cada registro dentro de una tabla. Ningún valor de llave primaria puede repetirse ni ser nulo.

- **Llave foránea (Foreign Key):** Es un campo en una tabla que hace referencia a la llave primaria de otra tabla, estableciendo una relación entre ambas tablas.

Por ejemplo, en una tabla "Pedidos", la columna "ClienteID" podría ser una llave foránea que hace referencia a la columna "ID" en la tabla "Clientes".

Ventajas de utilizar una base de datos

- 1 **Organización eficiente de la información:** En lugar de almacenar todos los datos en un solo archivo o tabla, las bases de datos permiten dividir los datos en múltiples tablas relacionadas, lo que facilita la gestión y evita redundancias.

- 2 **Acceso rápido a los datos:** Las bases de datos están optimizadas para permitir búsquedas y consultas rápidas. Gracias a las claves y a la estructura organizada de los datos, es posible recuperar información de manera eficiente, incluso en grandes volúmenes de datos.

- 3 **Seguridad y control de acceso:** Las bases de datos permiten gestionar el acceso a la información. Solo los usuarios autorizados pueden acceder a determinados datos, lo que protege la integridad y confidencialidad de la información.

- 4 **Integridad de los datos:** Las bases de datos relacionales garantizan que los datos estén consistentes y que las relaciones entre tablas se mantengan correctamente. Si un cliente es eliminado de la tabla "Clientes", no debería haber registros huérfanos en la tabla "Pedidos".

Relaciones en bases de datos

Las bases de datos relacionales permiten establecer relaciones entre tablas. Existen tres tipos principales de relaciones:

- **Uno a uno (1:1):** Cada registro en una tabla está asociado con un único registro en otra tabla. Por ejemplo, un empleado tiene un único carnet de identidad, y cada carnet de identidad pertenece a un solo empleado.

- **Uno a muchos (1:N):** Un registro en una tabla está relacionado con varios registros en otra tabla. Por ejemplo, un cliente puede hacer varios pedidos, pero cada pedido está asociado a un único cliente.

- **Muchos a muchos (N:N):** Varios registros de una tabla están relacionados con varios registros en otra tabla. Para implementar este tipo de relación, es común utilizar una tabla intermedia. Por ejemplo, un estudiante puede estar inscrito en varios cursos, y cada curso puede tener varios estudiantes. En este caso, se utiliza una tabla intermedia que relacione estudiantes con cursos.

En conclusión...

Las bases de datos son herramientas fundamentales para organizar y gestionar grandes volúmenes de información de manera eficiente. La capacidad de relacionar datos entre tablas permite una estructuración lógica que reduce la redundancia y mejora la consistencia.

Gracias a las bases de datos relacionales, es posible realizar consultas complejas, mantener la integridad de los datos y asegurar un acceso controlado y seguro. Ya sea en una pequeña empresa o en grandes sistemas corporativos, las bases de datos son esenciales para el funcionamiento adecuado de cualquier entorno basado en datos.

2. Consultas Básicas

2.1. SELECT & FROM

El comando SELECT en SQL se utiliza para **extraer datos** de una base de datos, permitiéndonos especificar qué columnas deseamos recuperar. Es uno de los comandos más fundamentales, ya que nos permite interactuar con la información almacenada.

Por otro lado, el comando FROM se utiliza para **indicar la tabla** de donde se obtendrán esos datos.

Juntos, SELECT y FROM forman la base de cualquier consulta que busca extraer información de una base de datos. Siempre van a ir de la mano, no puede existir una sin la otra.

```
SELECT "nombre", "apellido"  
FROM "clientes";
```

¿Qué es una Consulta o Query?

Una **consulta** (o **query**) es una instrucción SQL que se usa para interactuar con la base de datos.

Podemos usar consultas para obtener datos, actualizar registros, insertar o eliminar información.

Cómo abrir un nuevo script en DBeaver (con PostgreSQL)

Para empezar a escribir consultas SQL en **DBeaver** usando **PostgreSQL**, seguiremos estos pasos:

- 1 Abrir **DBeaver** y seleccionar la conexión de **PostgreSQL**.
- 2 Hacer clic derecho sobre la conexión y seleccionar **SQL Editor > New SQL Script**.

Esto abrirá una nueva ventana de script en la que podremos comenzar a escribir las consultas.

Buenas Prácticas en SQL

Es fundamental seguir **buenas prácticas** al escribir código SQL para asegurarnos de que el código sea legible y mantenga la coherencia en los resultados.

- 1 **Comandos en mayúsculas:** Escribir siempre las palabras clave de SQL (como SELECT, FROM, WHERE, etc.) en **mayúsculas** para diferenciarlas fácilmente de los nombres de tablas o columnas.

```
SELECT "nombre", "apellido"  
FROM "clientes";
```

- 2 **Finaliza las consultas con ";"**: Es crucial cerrar cada consulta con un punto y coma (;), especialmente si vamos a ejecutar múltiples consultas en el mismo script.

3 Utiliza comillas dobles para nombres de tablas y columnas: En PostgreSQL, es recomendable usar comillas dobles para nombrar tablas y columnas, ya que esto permite usar nombres que podrían tener espacios o mayúsculas.

4 Evitar usar * en SELECT: Aunque puedes usar `SELECT *` para seleccionar todas las columnas de una tabla, esto puede tener problemas:

- **Rendimiento:** Si la tabla tiene muchas columnas, seleccionar todas puede afectar el rendimiento.
- **Legibilidad:** No especificar qué columnas estás seleccionando puede hacer que la consulta sea difícil de entender y mantener.

En su lugar, es mejor especificar solo las columnas que necesitas.

Seleccionar Varias Columnas

Para seleccionar más de una columna, simplemente las enumeraremos por separado usando comas en la declaración `SELECT`. El orden de las columnas es importante, ya que determinará cómo se presentan los resultados.

Aquí, los datos se mostrarán primero con la columna "nombre", seguida de "apellido" y finalmente "correo". Si cambiamos el orden, los datos se mostrarán en el nuevo orden especificado

```
SELECT "nombre", "apellido", "correo"
FROM "clientes";
```

Comentarios en SQL

Los comentarios ayudan a documentar el código SQL, lo que es crucial para mantener una base de datos organizada y comprensible para otros desarrolladores (o para nosotros mismos en el futuro).

- **Comentarios de una línea:** Usaremos dos guiones (`--`) para comentar una línea.

```
-- Seleccionar todos los clientes
SELECT "nombre", "apellido"
FROM "clientes";
```

- **Comentarios multilínea:** Usaremos `/* */` para comentarios más largos o múltiples líneas.

```
/*
Esta consulta selecciona el nombre y apellido
de todos los clientes de la tabla "clientes"
*/
SELECT "nombre", "apellido"
FROM "clientes";
```

Guardar el Archivo

Recordemos guardar el archivo con el script SQL regularmente para evitar la pérdida del trabajo. En DBeaver, podemos guardar el archivo con **Ctrl + S** (**Cmd + S**) o desde el menú seleccionando **File > Save**.

2.2. WHERE

El comando WHERE en SQL se utiliza para **filtrar los datos** de una consulta, devolviendo solo las filas que cumplen con una condición específica. Es una herramienta poderosa que permite restringir los resultados de la consulta para obtener solo la información relevante

Por ejemplo, si queremos obtener solo los clientes que viven en una ciudad específica, podríamos usar el WHERE para aplicar este filtro:

```
SELECT "nombre", "apellido"
FROM "clientes"
WHERE "ciudad" = 'Madrid';
```

Utilización de Operadores con WHERE

El WHERE puede trabajar con varios **operadores** para definir las condiciones que deseamos aplicar.

Algunos de los operadores más comunes son:

- **Igual a (=)**: Filtra los resultados que son exactamente iguales a un valor específico
- **Mayor que (>), Menor que (<)**: Filtra resultados que son mayores o menores que un valor.

```
WHERE "edad" = 25;

WHERE "precio" > 100;
```

- **Mayor o igual que (>=), Menor o igual que (<=)**: Filtra los resultados que son mayores o iguales, o menores o iguales a un valor.
- **Distinto de (<>)**: Filtra los resultados que no son iguales a un valor.

```
WHERE "edad" >= 18;

WHERE "categoria" <> 'Electrónica';
```

Uso de YEAR() para Filtrar Fechas

Cuando trabajamos con campos de tipo **fecha**, a menudo es necesario filtrar por el **año** de la fecha.

Para esto, podemos utilizar la función YEAR() que extrae solo el año de un campo de fecha.

Por ejemplo, si queremos obtener todos los pedidos realizados en 2023, podríamos escribir.

```
SELECT "id_pedido", "fecha_pedido"
FROM "pedidos"
WHERE YEAR("fecha_pedido") = 2023;
```

Aquí, la función YEAR("fecha_pedido") extrae el año de cada valor en la columna "fecha_pedido" y compara el resultado con 2023.

Fechas en el WHERE: Uso de Comillas Simples

Al trabajar con fechas en el WHERE, es crucial recordar que las **fechas siempre deben ir entre comillas simples** (' '), no dobles, para que SQL las reconozca correctamente como literales de fecha.

En cambio, las comillas dobles se usan para nombres de columnas o tablas.

Por ejemplo, para obtener todos los pedidos realizados el 1 de enero de 2023

```
SELECT "id_pedido", "fecha_pedido"
FROM "pedidos"
WHERE "fecha_pedido" = '2023-01-01';
```

Aquí, el valor '2023-01-01' está entre comillas simples, lo que indica que es una fecha.

Si hubiéramos usado comillas dobles ("2023-01-01"), SQL trataría el valor como un **nombre de columna o tabla**, lo que resultaría en un error.

Texto en el WHERE: Uso de Comillas Simples

En SQL, las **comillas simples** (' ') se utilizan para representar **valores de texto o cadenas de caracteres** en las consultas. Esto es fundamental para que el motor de la base de datos entienda que el valor que estás proporcionando es una cadena literal y no el nombre de una columna o un comando SQL.

Si deseamos filtrar los resultados de una consulta según el valor de una columna de texto, las comillas simples son esenciales. Por ejemplo, si queremos seleccionar a todos los clientes cuyo país sea "España", usaríamos:

```
SELECT "nombre", "apellido"
FROM "clientes"
WHERE "pais" = 'España';
```

Uso de AND y OR

Los operadores **AND** y **OR** se utilizan para combinar múltiples condiciones en una sola cláusula WHERE.

- **AND**: Devuelve los resultados que cumplen **todas** las condiciones.
- **OR**: Devuelve los resultados que cumplen **al menos una** de las condiciones.

Si ahora quisiéramos seleccionar a los clientes que vivan en "España" o "México", escribiríamos:

```
SELECT "nombre", "apellido", "pais"
FROM "clientes"
WHERE "pais" = 'España'
OR "pais" = 'México';
```

Uso de IN y NOT IN

El operador **IN** se utiliza para comparar una columna con una lista de valores, simplificando consultas que requieren múltiples valores. **NOT IN** hace lo contrario: excluye los resultados que están en esa lista de valores.

Queremos **incluir** a los clientes que vivan en "España", "México" o "Argentina".

```
SELECT "nombre", "apellido", "pais"
FROM "clientes"
WHERE "pais" IN ('España', 'México', 'Argentina');
```

Queremos **excluir** a los clientes que viven en "España" o "México".

```
SELECT "nombre", "apellido", "pais"
FROM "clientes"
WHERE "pais" NOT IN ('España', 'México');
```

Uso de BETWEEN

El operador **BETWEEN** se utiliza para seleccionar valores dentro de un **rango** específico, incluyendo los valores extremos.

Queremos obtener los clientes cuya edad esté entre 25 y 40 años.

```
SELECT "nombre", "apellido", "edad"
FROM "clientes"
WHERE "edad" BETWEEN 25 AND 40;
```

Este operador también se puede usar con fechas. Si quisiéramos obtener los pedidos realizados entre el 1 de enero de 2023 y el 31 de diciembre de 2023, haríamos lo siguiente

```
SELECT "id_pedido", "fecha_pedido"
FROM "pedidos"
WHERE "fecha_pedido" BETWEEN '2023-01-01' AND '2023-12-31';
```

En conclusión...

La cláusula **WHERE** es fundamental en SQL porque nos permite filtrar los resultados de nuestras consultas basándonos en condiciones específicas. En esencia, el **WHERE** actúa como

un **filtro** que selecciona solo aquellos registros que cumplen con las condiciones que establecemos, ya sean numéricas, textuales o basadas en fechas.

- **Operadores numéricos:** =, >, <, >=, <=, <>, BETWEEN
- **Operadores para texto:** =, IN, NOT IN

2.3. AS

El concepto **AS** en SQL se utiliza para asignar un **alias** o nombre temporal a una columna o tabla dentro de una consulta. Esto es útil cuando queremos renombrar columnas para hacer que los resultados sean más claros y fáciles de entender, o cuando estamos trabajando con subconsultas o combinaciones de tablas y necesitamos distinguir los nombres

El AS nos permite:

- **Renombrar columnas o tablas:** Dar nombres más significativos o abreviados a columnas o tablas en nuestros resultados.
- **Clarificar consultas complejas:** Al asignar alias, podemos hacer que las consultas largas y complejas sean más fáciles de leer y entender.
- **Mejorar la presentación de los resultados:** Al cambiar nombres de columnas en los resultados de una consulta, se mejora la legibilidad de los datos.

Alias para columnas

Supongamos que queremos cambiar el nombre de una columna en el resultado de una consulta.

```
SELECT "nombre", "apellido", "edad" AS "años"
FROM "clientes";
```

Alias para tablas

También podemos usar AS para asignar un alias a una tabla, especialmente útil cuando trabajamos con varias tablas o con combinaciones

```
SELECT "c"."nombre"
FROM "clientes" AS "c"
```

En conclusión...

El uso de **AS** nos permite simplificar y personalizar nuestras consultas en SQL. Es una herramienta que facilita la comprensión y organización de los resultados, mejorando tanto la legibilidad del código como la presentación de los datos.

3. Consultas de Agregación

Las **funciones de agregación** en SQL son herramientas que nos permiten realizar cálculos sobre un conjunto de valores en lugar de sobre cada fila individual.

Estas funciones toman múltiples filas como entrada y devuelven un solo valor como resultado, ayudándonos a resumir grandes volúmenes de datos de manera eficiente.

Las funciones de agregación más comunes incluyen **SUM**, **COUNT**, **AVG**, **MIN**, y **MAX**.

3.1. MIN & MAX

- **MIN**: Esta función devuelve el valor más pequeño de una columna, ya sea numérica, de texto o de fecha.

```
SELECT MIN("salario") AS "salario_minimo"  
FROM "empleados";
```

- **MAX**: De manera similar, **MAX** devuelve el valor más grande de una columna.

```
SELECT MAX("salario") AS "salario_maximo"  
FROM "empleados";
```

Errores comunes al combinar funciones de agregación y columnas completas

Un error frecuente ocurre cuando intentamos usar funciones de agregación junto con columnas no agrupadas en la misma consulta sin aplicar condiciones de agrupamiento (sin utilizar **GROUP BY**).

En estos casos, si queremos aplicar una función como **MIN** o **MAX** pero también queremos mostrar otras columnas, el motor SQL nos arrojará un error.

```
SELECT "nombre", MAX("salario")  
FROM "empleados";
```

El error ocurre porque no estamos indicando cómo agrupar los datos. La función de agregación está diseñada para devolver un único valor, pero también estamos intentando seleccionar una columna completa ("nombre"), lo cual no es coherente.

Uso de MIN y MAX con fechas

Las funciones **MIN** y **MAX** son muy útiles para trabajar con **fechas**.

Nos permiten, por ejemplo, encontrar la fecha más antigua o la más reciente en un conjunto de datos

Uso de MIN y MAX con texto

Aunque MIN y MAX son más comunes con números y fechas, también se pueden usar con **valores de texto**. SQL evalúa el texto en **orden alfabético**.

Por ejemplo, MIN devolverá el valor que está primero alfabéticamente, y MAX el que está último.

En conclusión...

Las funciones de agregación como **MIN** y **MAX** nos permiten obtener el valor más pequeño o grande en una columna de datos numéricos, fechas o texto.

Sin embargo, es importante tener en cuenta que no podemos combinarlas con la selección de columnas completas sin algún tipo de agrupación.

Además, estas funciones son sumamente versátiles y útiles en una amplia variedad de contextos, desde la búsqueda del salario más alto hasta la identificación de fechas límite o el orden alfabético en una lista.

3.2. COUNT & SUM

En **SQL**, las funciones agregadas como **COUNT** y **SUM** son herramientas esenciales para realizar análisis de datos, permitiendo resumir grandes volúmenes de información

COUNT

La función COUNT cuenta la cantidad de filas o valores en una columna o conjunto de datos.

Dependiendo de cómo la utilicemos, COUNT puede contar todas las filas, solo aquellas que no son NULL, o las filas que contienen valores únicos.

Este ejemplo devuelve el **número total de filas** en la tabla empleados, sin importar si hay valores NULL o no.

```
SELECT COUNT(*)
FROM "empleados";
```

Tipos de COUNT

- 1 COUNT(*):** Cuenta todas las filas de la tabla, incluidas aquellas con valores NULL en cualquier columna.
- 2 COUNT(columna):** Cuenta solo las filas donde la columna **no sea NULL**. Si la columna contiene valores NULL, no los cuenta.
- 3 COUNT(DISTINCT (columna)):** Cuenta solo los valores **únicos** en una columna, omitiendo duplicados y excluyendo NULL por defecto.
- 4 COUNT(columna IS NOT NULL):** Se puede usar en conjunto con un filtro explícito para asegurarse de contar solo los valores que no son NULL.

```
-- Total de empleados
SELECT COUNT(*) AS "total_empleados"
FROM "empleados";

-- Número de empleados con edad registrada
SELECT COUNT("edad") AS "empleados_con_edad"
FROM "empleados";

-- Número de departamentos únicos
SELECT COUNT(DISTINCT "departamento") AS "departamentos_unicos"
FROM "empleados";

-- Número de empleados con salario registrado
SELECT COUNT(*) AS "empleados_con_salario"
FROM "empleados"
WHERE "salario IS NOT NULL;
```

SUM

La función SUM se utiliza para calcular la suma total de los valores en una columna específica.

Solo puede aplicarse a **columnas numéricas**

Esto suma todos los valores de la columna salario en la tabla empleados. Si alguna fila tiene un valor NULL en la columna salario, ese valor será ignorado.

```
SELECT SUM("salario")
FROM "empleados";
```

Consideraciones importantes sobre SUM

- 1 NULL en SUM:** Al sumar valores, si una fila tiene un valor NULL, simplemente se ignora en el cálculo. SUM no cuenta los valores NULL y no los trata como 0
- 2 SUM(DISTINCT columna):** Podemos usar DISTINCT para sumar solo los valores únicos en una columna.

```
-- Suma total de los salarios de los empleados
SELECT SUM("salario") AS "suma_salarios"
FROM "empleados";

-- Suma total de los salarios, excluyendo valores NULL
SELECT SUM("salario") AS "suma_salarios_validos"
FROM "empleados"
WHERE "salario" IS NOT NULL;

-- Suma de los salarios únicos
SELECT SUM(DISTINCT "salario") AS "suma_salarios_unicos"
FROM "empleados";
```

En conclusión...

- **COUNT** cuenta el número de filas o valores, y tiene variantes como COUNT(*), que cuenta todas las filas, y COUNT(columna), que excluye los valores NULL.
- **SUM** calcula la suma total de los valores en una columna numérica, ignorando los valores NULL. También permite sumar valores únicos mediante SUM(DISTINCT columna).

Ambas funciones son ampliamente usadas para resumir datos, ya sea para contar registros o calcular el total de valores numéricos en una tabla.

3.3. AVG, STDDEV, VARIANCE

Las funciones **AVG**, **STDDEV** y **VARIANCE** son funciones agregadas en SQL que nos permiten realizar análisis estadísticos sobre los datos numéricos en nuestras tablas.

- **AVG** calcula el **promedio** de un conjunto de valores.
- **STDDEV** calcula la **desviación estándar**, que mide la dispersión de los datos.
- **VARIANCE** mide **la variabilidad o el grado de dispersión** de un conjunto de datos.

Estas funciones nos ayudan a obtener una visión más profunda de nuestros datos, permitiéndonos identificar patrones y distribuciones dentro de un conjunto de registros.

AVG

La función **AVG** nos permite calcular el promedio de los valores de una columna numérica.

Ignora los valores NULL y divide la suma total de los valores por el número de valores no nulos.

En este ejemplo calculamos el promedio de los salarios de los empleados. Los valores NULL en la columna "salario" no se tendrán en cuenta en el cálculo.

```
SELECT AVG("salario")
FROM "empleados";
```

ROUND

A menudo queremos redondear el resultado de **AVG** a un número específico de decimales. La función **ROUND** nos permite hacerlo.

En este caso, redondeamos el promedio de los salarios a 2 decimales.

```
SELECT ROUND(AVG("salario"), 2) AS "promedio_redondeado"
FROM "empleados";
```

STDDEV

La función **STDDEV** nos ayuda a calcular la **desviación estándar** de los datos en una columna. La desviación estándar mide cuánto se alejan los valores de la media (promedio).

Cuanto mayor sea el valor de la desviación estándar, mayor será la dispersión de los datos.

Este ejemplo calcula la dispersión de los salarios alrededor del promedio.

```
SELECT STDDEV("salario") AS "desviacion_salarial"
FROM "empleados";
```

VARIANCE

La función **VARIANCE** calcula la **varianza**, que es el cuadrado de la desviación estándar. Nos indica la variabilidad o dispersión de los datos de forma más directa.

Aquí calculamos la varianza de los salarios, lo que nos ayuda a entender cómo se distribuyen los salarios en torno al promedio.

```
SELECT VARIANCE("salario") AS "varianza_salarial"
FROM "empleados";
```

En conclusión...

- **AVG** nos proporciona el **promedio** de los valores numéricos.
- **ROUND** se usa para **redondear** los resultados de funciones como **AVG**.
- **STDDEV** mide la **desviación estándar**, indicándonos la dispersión de los datos.
- **VARIANCE** nos muestra la **variabilidad** de los datos, siendo el cuadrado de la desviación estándar.

Con estas funciones, podemos analizar los datos desde una perspectiva estadística, obteniendo una visión más completa sobre la distribución y consistencia de nuestros valores

3.4. CONCAT

La función **CONCAT** en SQL se utiliza para **unir** o **concatenar** varias cadenas de texto en una sola.

Es muy útil cuando queremos combinar el contenido de diferentes columnas o valores dentro de una consulta, creando una sola cadena de salida.

Sintaxis básica:

```
SELECT CONCAT(columna1, columna2, ...) AS "resultado"
FROM tabla;
```

Por ejemplo, vamos a unir la columna "nombre" con un espacio (' ') y la columna "apellido", de modo que el resultado sea el nombre completo del empleado.

```
SELECT CONCAT("nombre", ' ', "apellido") AS "nombre_completo"
FROM "empleados";
```

Valores NULL

Si alguna de las columnas contiene valores NULL, la función CONCAT lo trata como una cadena vacía. Por ejemplo, concatenar NULL con "Texto" resultará en solo "Texto".

En este ejemplo no se generará error, simplemente omitirá el NULL.

```
SELECT CONCAT("nombre", "apellido", NULL) AS "resultado"
FROM "empleados";
```

En conclusión...

La función **CONCAT** es esencial cuando necesitamos combinar múltiples columnas o textos en una sola cadena.

Nos permite formatear resultados, mostrar información más clara, o crear nuevos valores a partir de los datos existentes en nuestras tablas.

Es simple de usar y altamente flexible, pero hay que tener en cuenta cómo maneja los valores NULL para evitar resultados inesperados

4. Consultas de Ordenación y Agrupación

4.1. ORDER BY

La cláusula **ORDER BY** en SQL se utiliza para **ordenar** los resultados de una consulta en función de los valores de una o más columnas. Nos permite controlar si los datos deben aparecer en orden **ascendente** (ASC) o **descendente** (DESC)

Sintaxis básica

```
SELECT columna1, columna2
FROM tabla
ORDER BY columna1 [ASC|DESC];
```

Este ejemplo ordena los empleados por su salario en orden descendente, de mayor a menor.

```
SELECT "nombre", "salario"
FROM "empleados"
ORDER BY "salario" DESC;
```

Ordenar por más de una columna

Podemos ordenar los resultados por más de una columna especificando varias columnas en la cláusula ORDER BY.

El orden de prioridad será de izquierda a derecha.

Este ejemplo ordena primero por el "departamento" en orden ascendente, y dentro de cada departamento, por "salario" en orden descendente.

```
SELECT "nombre", "departamento", "salario"
FROM "empleados"
ORDER BY "departamento" ASC, "salario" DESC;
```

ORDER BY con WHERE

Podemos combinar la cláusula ORDER BY con WHERE para filtrar los datos y luego ordenarlos. La cláusula WHERE restringe los resultados que queremos seleccionar, y luego ORDER BY organiza estos resultados filtrados.

Aquí, primero filtramos los empleados con un salario mayor a 50,000 y luego ordenamos esos empleados en orden ascendente por su salario.

```
SELECT "nombre", "salario"
FROM "empleados"
WHERE "salario" > 50000
ORDER BY "salario" ASC;
```

Orden de ejecución de SQL y alias en WHERE (cómo lee SQL las consultadas)

El orden de ejecución de las cláusulas en una consulta SQL es importante para entender por qué no podemos utilizar alias creados en el SELECT dentro de la cláusula WHERE.

1 FROM: Primero se selecciona la tabla o tablas.

2 WHERE: Luego se filtran los datos.

3 SELECT: A continuación, se seleccionan las columnas, y se aplican alias si es necesario.

4 ORDER BY: Finalmente, se ordenan los resultados.

Dado que el alias se define en la cláusula SELECT, y la cláusula WHERE se ejecuta **antes** del SELECT, no podemos referirnos a alias en WHERE.

En este caso, el alias "nuevo_salario" no es reconocido en el WHERE, ya que este se ejecuta antes de que se cree el alias en el SELECT.

```
SELECT "salario" * 1.1 AS "nuevo_salario"
FROM "empleados"
WHERE "nuevo_salario" > 60000; -- Esto generará un error.
```

En conclusión...

La cláusula **ORDER BY** es fundamental para organizar los resultados de nuestras consultas SQL, y podemos aplicarla a una o varias columnas, incluso combinada con WHERE.

Es importante recordar el orden de ejecución de SQL, ya que nos ayuda a entender por qué los alias no pueden usarse en WHERE, pero sí funcionan en ORDER BY y en otras partes de la consulta.

4.2. LIMIT & OFFSET

La cláusula **LIMIT** en SQL se utiliza para restringir el número de filas devueltas por una consulta, y **OFFSET** nos permite omitir un número específico de filas antes de comenzar a devolver los resultados.

Estas dos cláusulas son útiles cuando trabajamos con grandes conjuntos de datos y queremos paginar los resultados o controlar la cantidad de información que procesamos.

LIMIT

El número especificado en LIMIT será la cantidad de filas que la consulta devolverá.

Este ejemplo selecciona las primeras 10 filas de la tabla "empleados".

```
SELECT "nombre", "salario"
FROM "empleados"
LIMIT 10;
```

Sintaxis básica de LIMIT

```
SELECT columna1, columna2
FROM tabla
LIMIT número_filas;
```

LIMIT con ORDER BY

LIMIT es frecuentemente combinado con la cláusula **ORDER BY** para obtener los primeros o últimos registros según el criterio de orden.

Este ejemplo selecciona los 5 empleados con el salario más alto, ya que hemos ordenado los salarios en orden descendente con ORDER BY y luego aplicado LIMIT para devolver solo los primeros 5.

```
SELECT "nombre", "salario"
FROM "empleados"
ORDER BY "salario" DESC
LIMIT 5;
```

LIMIT, WHERE y ORDER BY

Podemos combinar **LIMIT** con **WHERE** y **ORDER BY** para filtrar, ordenar y luego limitar los resultados devueltos por la consulta.

Este ejemplo primero filtra los empleados con salarios mayores a 50,000, luego los ordena en orden descendente, y finalmente devuelve solo los 3 empleados con los salarios más altos

```
SELECT "nombre", "salario"
FROM "empleados"
WHERE "salario" > 50000
ORDER BY "salario" DESC
LIMIT 3;
```

OFFSET

OFFSET nos permite omitir un número de filas antes de comenzar a devolver los resultados. Esto es útil para implementar **paginación**, donde queremos mostrar los resultados por bloques o páginas.

En este ejemplo, se omiten las primeras 10 filas y se seleccionan las siguientes 5 filas. Es decir, los empleados del puesto 11 al 15 en el ranking de salarios descendente.

```
SELECT "nombre", "salario"
FROM "empleados"
ORDER BY "salario" DESC
LIMIT 5 OFFSET 10;
```

Orden de ejecución de SQL

Entender el orden en que se ejecutan las partes de una consulta SQL es crucial para comprender por qué ciertas cláusulas deben aparecer en un orden específico:

- 1 **FROM:** Primero se selecciona la tabla o tablas.
- 2 **WHERE:** Luego se filtran los datos.
- 3 **SELECT:** A continuación, se seleccionan las columnas, y se aplican alias si es necesario.
- 4 **ORDER BY:** Finalmente, se ordenan los resultados.
- 5 **LIMIT y OFFSET:** Finalmente, se limitan y/o omiten las filas devueltas según lo especificado.

En conclusión...

LIMIT y OFFSET son herramientas poderosas cuando queremos controlar cuántas filas procesamos y cómo paginar los resultados.

Combinados con ORDER BY y WHERE, nos permiten hacer consultas eficientes en grandes volúmenes de datos.

Entender el orden de ejecución de SQL es clave para que las consultas sean efectivas y correctas.

4.3. GROUP BY & HAVING

El uso de GROUP BY en SQL nos permite agrupar filas que tienen valores iguales en una o más columnas, lo que es esencial cuando trabajamos con **funciones de agregación** como SUM(), COUNT(), AVG(), MIN(), y MAX().

Por otro lado, el **HAVING** se utiliza para aplicar filtros en los resultados de una consulta agregada, de forma similar a cómo usamos el **WHERE** para filtrar en consultas sin agregación.

¿Cuándo usamos WHERE y cuándo usamos HAVING?

- **WHERE:** Se utiliza **antes** de aplicar el GROUP BY para filtrar las filas **individuales** que van a ser agrupadas. Es decir, filtramos los datos **antes** de que se realice la agregación. Se puede utilizar con cualquier tipo de condición que actúe sobre columnas sin agregar.
- **HAVING:** Se usa **después** de aplicar el GROUP BY para filtrar los grupos resultantes de la agregación. Es útil cuando queremos aplicar una condición a los resultados de las funciones de agregación (por ejemplo, filtrar grupos que tienen un valor agregado específico).

Sintaxis básica de GROUP BY y HAVING

```
SELECT columna1, columna2, AGG_FUNC(columna3)
FROM nombre_tabla
WHERE condicion_filas_individuales
GROUP BY columna1, columna2
HAVING condicion_agrupada;
```

Orden de ejecución de SQL con GROUP BY y HAVING

- 1 **FROM:** Se selecciona la tabla o tablas desde las cuales obtenemos los datos.
- 2 **WHERE:** Se filtran las filas individuales de acuerdo con las condiciones que no implican funciones de agregación.
- 3 **GROUP BY:** Se agrupan las filas en grupos según las columnas especificadas.
- 4 **Funciones de agregación:** Se calculan las funciones de agregación, como SUM(), COUNT(), etc., para cada grupo
- 5 **HAVING:** Se aplican las condiciones para filtrar grupos basados en los resultados de las funciones de agregación.
- 6 **SELECT:** Finalmente, se seleccionan las columnas o agregaciones que queremos mostrar.

Si quisiéramos saber el número total de ventas por producto, pero solo para aquellas ventas que superaron los 50 euros, podríamos utilizar WHERE para filtrar las filas individuales antes de agrupar

```
SELECT "producto", COUNT(*) AS "num_ventas"
FROM "ventas"
WHERE "monto" > 50
GROUP BY "producto";
```

Si en lugar de filtrar las filas individuales, quisiéramos obtener productos que en total hayan tenido más de 10 ventas, usaríamos **HAVING**

```
SELECT "producto", COUNT(*) AS "num_ventas"
FROM "ventas"
GROUP BY "producto"
HAVING COUNT(*) > 10;
```

En conclusión...

- Usamos **WHERE** para filtrar filas antes de realizar el agrupamiento, mientras que **HAVING** se utiliza para filtrar los resultados después de aplicar las funciones de agregación.
- **WHERE** se usa con columnas simples, mientras que **HAVING** se usa para filtrar con funciones agregadas como COUNT(), SUM(), etc.

5. Relaciones entre tablas

Los **joins** en SQL se utilizan para combinar filas de dos o más tablas basadas en una condición relacionada entre ellas.

Los joins son cruciales cuando trabajamos con bases de datos relacionales, ya que nos permiten extraer y relacionar datos de múltiples tablas.

Tipos más comunes de JOINS

Existen varios tipos de joins que sirven para diferentes propósitos:

- 1 **INNER JOIN**: Devuelve las filas cuando hay una coincidencia en ambas tablas.
- 2 **LEFT JOIN**: Devuelve todas las filas de la tabla de la izquierda y las filas coincidentes de la tabla de la derecha. Si no hay coincidencia, se rellena con NULL en la tabla de la derecha.
- 3 **RIGHT JOIN**: Similar al LEFT JOIN, pero devuelve todas las filas de la tabla de la derecha y las coincidencias de la izquierda. Si no hay coincidencia, se rellena con NULL en la izquierda.
- 4 **CROSS JOIN**: Devuelve el producto cartesiano de las dos tablas, es decir, combina cada fila de la primera tabla con cada fila de la segunda, sin ninguna condición.
- 5 **FULL JOIN** (o **FULL OUTER JOIN**): Devuelve todas las filas cuando hay una coincidencia en al menos una de las tablas, rellenando con NULL cuando no haya coincidencias.

Orden de Ejecución de un Join

- 1 **FROM**: Se seleccionan las tablas.
- 2 **ON**: Se especifica la condición que relaciona las tablas.
- 3 **JOIN**: Se combina el contenido de las tablas según el tipo de join especificado.
- 4 **WHERE**: Se filtran las filas resultantes del join.
- 5 **SELECT**: Se seleccionan las columnas de las tablas combinadas.
- 6 **ORDER BY** y **LIMIT**: Se ordenan y limitan los resultados finales.

5.1. INNER JOIN

El **INNER JOIN** es uno de los joins más comunes y se utiliza para combinar filas de dos tablas cuando hay una coincidencia en ambas según una condición especificada.

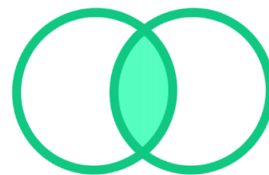
Esto significa que el **INNER JOIN** solo devuelve las filas donde los valores coinciden en ambas tablas, excluyendo cualquier fila que no tenga coincidencia en alguna de las tablas.

IdCliente	Nombre	Apellidos	Calle
1	Lola	López	Calle Venus
2	Marta	García	Calle Estrella
3	Roberto	Rosira	Calle Luna
4	Mario	Pascual	Calle Marte

IdFactura	Cantidad	Precio	IdCliente
2222	2	12	1
0000	1	150	1
3333	10	2683	4
4444	3	15	1

Resultado INNER JOIN

IdCliente	Nombre	Apellidos	Calle	IdFactura	Cantidad	Precio
1	Lola	López	Calle Venus	2222	2	12
1	Lola	López	Calle Venus	0000	1	150
1	Lola	López	Calle Venus	4444	3	15
4	Mario	Pascual	Calle Marte	3333	10	2683



INNER JOIN

Sintaxis del INNER JOIN

```
SELECT tabla1.columna1, tabla2.columna2
FROM tabla1
INNER JOIN tabla2
ON tabla1.columna_relacionada = tabla2.columna_relacionada;
```

Supongamos que tenemos dos tablas: "**empleados**" y "**departamentos**". Queremos obtener el nombre del empleado y el nombre del departamento en el que trabaja, pero solo para aquellos empleados que tienen un departamento asignado (es decir, hay una coincidencia en ambas tablas).

Este INNER JOIN devolverá solo aquellos empleados que tienen un departamento asignado, excluyendo a los empleados que no estén vinculados a un departamento

```
SELECT "e"."nombre", "d"."nombre"
FROM "empleados" AS "e"
INNER JOIN "departamentos" AS "d"
ON "e"."departamento_id" = "d"."id";
```

En conclusión...

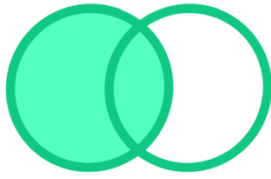
El **INNER JOIN** es una herramienta esencial cuando necesitamos combinar datos de varias tablas, asegurándonos de que solo obtenemos los registros donde hay coincidencias entre ambas tablas.

A diferencia de los **OUTER JOINS**, los **INNER JOINS** excluyen las filas sin correspondencia, lo que los hace útiles cuando queremos trabajar solo con datos completamente relacionados.

5.2. LEFT JOIN

El **LEFT JOIN** es un tipo de join en SQL que devuelve todas las filas de la tabla de la izquierda (la primera tabla mencionada en la consulta), junto con las filas coincidentes de la tabla de la derecha (segunda tabla mencionada).

Si no hay coincidencias en la tabla de la derecha, las columnas correspondientes se rellenan con NULL. Esto lo convierte en una herramienta útil cuando queremos asegurarnos de obtener todas las filas de una tabla, independientemente de si tienen una coincidencia en la otra



LEFT JOIN

IdCliente	Nombre	Apellidos	Calle
1	Lola	López	Calle Venus
2	Marta	García	Calle Estrella
3	Roberto	Romira	Calle Luna
4	Mario	Pascual	Calle Marte

IdFactura	Cantidad	Precio	IdCliente
2222	2	12	1
0000	1	150	1
3333	10	2683	4
4444	3	15	1
8888	2	214	7

Resultado LEFT JOIN

IdCliente	Nombre	Apellidos	Calle	IdFactura	Cantidad	Precio
1	Lola	López	Calle Venus	2222	2	12
1	Lola	López	Calle Venus	0000	1	150
1	Lola	López	Calle Venus	4444	3	15
2	Marta	García	Calle Estrella	NaN	NaN	NaN
3	Roberto	Romira	Calle Luna	NaN	NaN	NaN
4	Mario	Pascual	Calle Marte	3333	10	2683

Sintaxis del LEFT JOIN

```
SELECT tabla1.columna1, tabla2.columna2
FROM tabla1
LEFT JOIN tabla2
ON tabla1.columna_relacionada = tabla2.columna_relacionada;
```

Supongamos que tenemos dos tablas: "empleados" y "departamentos". Queremos obtener el nombre de todos los empleados y el nombre de su departamento, pero no todos los empleados están asignados a un departamento.

```
SELECT "e"."nombre", "d"."nombre"
FROM "empleados" AS "e"
LEFT JOIN "departamentos" AS "d"
ON "e"."departamento_id" = "d"."id";
```

En este caso, la consulta devolverá todos los empleados, y aquellos que no tengan un departamento asignado tendrán un valor NULL en la columna del nombre del departamento.

En conclusión...

El **LEFT JOIN** es ideal cuando necesitamos todos los registros de una tabla principal y no nos importa si la otra tabla tiene o no datos relacionados.

Es especialmente útil en informes o consultas donde queremos obtener un conjunto completo de datos, sin perder filas que no tengan coincidencias en otras tablas. (ver video)

5.3. RIGHT JOIN

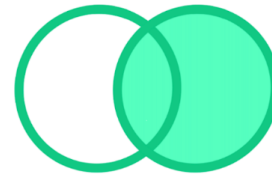
El RIGHT JOIN devuelve todas las filas de la **tabla derecha** y las filas coincidentes de la **tabla izquierda**. Si no hay coincidencia en la tabla izquierda, los valores serán NULL para las columnas de esa tabla.

IdCliente	Nombre	Apellidos	Calle
1	Lola	López	Calle Venus
2	Marta	García	Calle Estrella
3	Roberto	Romira	Calle Luna
4	Mario	Pascual	Calle Marte

IdFactura	Cantidad	Precio	IdCliente
2222	2	12	1
0000	1	150	1
3333	10	2683	4
4444	3	15	1
8888	2	214	7

Resultado RIGHT JOIN

IdCliente	Nombre	Apellidos	Calle	IdFactura	Cantidad	Precio	IdCliente
1	Lola	López	Calle Venus	2222	2	12	1
1	Lola	López	Calle Venus	0000	1	150	1
4	Mario	Pascual	Calle Marte	3333	10	2683	4
1	Lola	López	Calle Venus	4444	3	15	1
NaN	NaN	NaN	NaN	8888	2	214	7



RIGHT JOIN

Sintaxis básica

```
SELECT tabla1.columna, tabla2.columna
FROM tabla1
RIGHT JOIN tabla2
ON tabla1.columna_comun = tabla2.columna_comun;
```

Supongamos que queremos obtener todos los departamentos de nuestra empresa, independientemente de si tienen empleados asignados o no. Sin embargo, en lugar de poner en el FROM la tabla "departamentos" ponemos la tabla "empleados"

```
SELECT empleados.nombre, departamentos.nombre_departamento
FROM empleados
RIGHT JOIN departamentos
ON empleados.departamento_id = departamentos.id;
```

En conclusión...

El RIGHT JOIN es útil cuando queremos asegurarnos de que **todas las filas de la tabla derecha** se incluyan en el resultado de nuestra consulta, independientemente de si existen coincidencias en la tabla izquierda. Este tipo de unión es ideal cuando deseamos priorizar los datos de la tabla derecha y aún así visualizar información relacionada, si existe, en la tabla izquierda.

En resumen, el RIGHT JOIN garantiza que ninguna fila de la tabla derecha se excluya, rellenando con NULL los valores faltantes en la tabla izquierda cuando no hay coincidencias. Es una herramienta importante en casos donde debemos analizar todas las filas de una tabla secundaria sin perder información relevante por falta de coincidencias en la principal.

5.4. CROSS JOIN

El **CROSS JOIN** en SQL es un tipo de join que devuelve el **producto cartesiano** de dos tablas. Esto significa que cada fila de la primera tabla se combina con cada fila de la segunda tabla, sin ninguna condición que relacione las tablas.

Este tipo de join generalmente no se utiliza para relaciones entre datos, sino en situaciones donde queremos todas las combinaciones posibles entre dos conjuntos de datos.



Sintaxis del CROSS JOIN

```
SELECT tabla1.columna1, tabla2.columna2
FROM tabla1
CROSS JOIN tabla2;
```

Supongamos que tenemos dos tablas: "**productos**" y "**colores**". Queremos generar una lista de todas las combinaciones posibles de productos y colores disponibles.

Si la tabla "**productos**" tiene 5 filas y la tabla "**colores**" tiene 3 filas, el resultado del **CROSS JOIN** será $5 * 3 = 15$ combinaciones.

```
SELECT "p"."nombre", "c"."color"
FROM "productos" AS "p"
CROSS JOIN "colores" AS "c";
```

El **CROSS JOIN** genera un número de filas igual al producto del número de filas en las dos tablas involucradas. No hay condiciones para unir las tablas, por lo que cada fila de la primera tabla se empareja con cada fila de la segunda tabla.

En conclusión...

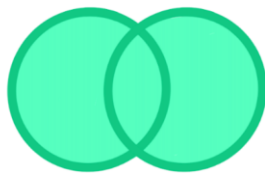
El **CROSS JOIN** es una herramienta poderosa para crear todas las combinaciones posibles entre dos tablas. Sin embargo, es importante usarlo con precaución, ya que el tamaño del resultado puede volverse inmanejable si las tablas involucradas contienen muchas filas.

5.5. FULL JOIN

El **FULL JOIN** en SQL es una combinación de un **LEFT JOIN** y un **RIGHT JOIN**, lo que significa que devuelve todas las filas cuando hay una coincidencia en cualquiera de las tablas.

Si no hay coincidencia, las filas de las tablas se incluyen en el resultado con valores nulos para las columnas que no tienen coincidencia.

El **FULL JOIN** es útil cuando queremos incluir todas las filas de ambas tablas, tanto si tienen correspondencia como si no.



FULL JOIN

IdCliente	Nombre	Apellidos	Calle
1	Lola	López	Calle Venus
2	Marta	García	Calle Estrella
3	Roberto	Romira	Calle Luna
4	Mario	Pascual	Calle Marte
5	Simeón	Urso	Calle Júpiter

IdFactura	Cantidad	Precio	IdCliente
2222	2	12	1
0000	1	150	1
3333	10	2683	4
4444	3	15	1
8888	2	214	7

Resultado FULL JOIN

IdCliente	Nombre	Apellidos	Calle	IdFactura	Cantidad	Precio	IdCliente
1	Lola	López	Calle Venus	2222	2	12	1
1	Lola	López	Calle Venus	0000	1	150	1
4	Mario	Pascual	Calle Marte	3333	10	2683	4
1	Lola	López	Calle Venus	4444	3	15	1
NaN	NaN	NaN	NaN	8888	2	214	7
2	Marta	García	Calle Estrella	NaN	NaN	NaN	NaN
3	Roberto	Romira	Calle Luna	NaN	NaN	NaN	NaN
5	Simeón	Urso	Calle Júpiter	NaN	NaN	NaN	NaN

Sintaxis del FULL JOIN

```
SELECT tabla1.columna1, tabla2.columna2
FROM tabla1
FULL JOIN tabla2
ON tabla1.columna_comun = tabla2.columna_comun;
```

Supongamos que tenemos dos tablas: "**clientes**" y "**pedidos**". La tabla "**clientes**" tiene información de los clientes, y la tabla "**pedidos**" tiene información sobre las órdenes de los clientes.

```
SELECT "c"."nombre", "p"."pedido_id"
FROM "clientes" AS "c"
FULL JOIN "pedidos" AS "p"
ON "c"."cliente_id" = "p"."cliente_id";
```

Uso del FULL JOIN

El **FULL JOIN** es útil cuando queremos asegurarnos de que obtenemos todos los datos posibles de ambas tablas, independientemente de si hay una coincidencia entre ellas.

Es especialmente útil cuando necesitamos combinar datos incompletos de varias fuentes y queremos ver tanto los datos coincidentes como los faltantes.

En conclusión...

El **FULL JOIN** es una herramienta poderosa cuando necesitamos combinar tablas y obtener tanto las coincidencias como las no coincidencias entre ellas.

Nos permite analizar tanto los elementos que tienen relación como aquellos que no la tienen, ofreciendo una vista completa de los datos de ambas tablas.

6. Subconsultas

Una **subconsulta** es una consulta anidada dentro de otra consulta.

Las subconsultas permiten realizar consultas más complejas al utilizar el resultado de una consulta interna dentro de otra.

6.1. Subconsultas con Where

Una de las formas más comunes de utilizar subconsultas es con la cláusula **WHERE**, para filtrar resultados basándose en los datos obtenidos de otra consulta

Supongamos que queremos listar todos los productos que pertenecen a categorías activas.

Tenemos dos tablas: productos y categorías. Primero obtenemos las categorías activas y luego filtramos los productos con esa información:

Estructura básica de una subconsulta con WHERE

```
SELECT "nombre", "precio"
FROM "productos"
WHERE "categoria_id" IN (
  SELECT "id"
  FROM "categorías"
  WHERE "activa" = TRUE
);
```

```
SELECT columna1, columna2
FROM tabla1
WHERE columnaX IN (
  SELECT columnaY
  FROM tabla2
  WHERE condición
);
```

[Copiar código](#)

Buenas prácticas

1 Optimización: Las subconsultas pueden ralentizar la ejecución de las consultas si no se optimizan correctamente, especialmente en bases de datos con grandes volúmenes de datos. Si es posible, es recomendable evaluar el uso de **joins** o vistas para mejorar el rendimiento.

2 Legibilidad: Asegurémonos de que las subconsultas no sean demasiado complejas. Podemos desglosar consultas largas o anidadas en varias partes para mejorar la claridad.

3 Uso de operadores: En la cláusula WHERE de una subconsulta, podemos utilizar varios operadores, como IN, NOT IN, =, >, <, dependiendo del tipo de comparación que queramos hacer.

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- Comencemos obteniendo los artistas que tengan más de un álbum

```

SELECT "ArtistId", "Name" #Seleccionamos los artistas
FROM "Artist" #De la tabla "Artist"
WHERE "ArtistId" IN ( #Cuyo "ArtistID" se encuentre...
    SELECT "ArtistId" #En la selección de "ArtistID"
    FROM "Album" #De la tabla Album
    GROUP BY "ArtistId" #Que al agrupar por "ArtistID"
    HAVING COUNT("AlbumId") > 1 #El conteo de "AlbumID" sea mayor de 1
);

```

- Ahora obtengamos aquellas canciones que tengan un precio mayor que la media

```

SELECT "TrackId", "Name", "UnitPrice" #Seleccionamos las canciones
FROM "Track" #De la tabla "Track"
WHERE "UnitPrice" > ( #Cuyo precio unitario sea mayor
    SELECT AVG("UnitPrice") #Que la media de todos los precios unitarios
    FROM "Track"
);

```

- Hagamos varias subconsultas anidadas. Obtengamos los clientes los cuales realizaron compras por un valor mayor al promedio de compras de todos los clientes

```

SELECT "CustomerId", "FirstName", "LastName" #Seleccionemos los clientes
FROM "Customer" #De la tabla Customer
WHERE "CustomerId" IN ( #Cuyo CustomerID se encuentre en...
    SELECT "CustomerId" #La selección de CustomerID
    FROM "Invoice" #De la tabla "Invoice"
    GROUP BY "CustomerId" #Agrupando por "CustomerID"
    HAVING SUM("Total") > ( #Cuya suma del total sea mayor que
        SELECT AVG("Total") #La media de total de la tabla Invoice
        FROM "Invoice"
    )
);

```

En conclusión...

Las subconsultas con WHERE son una herramienta poderosa para realizar filtros complejos en SQL, permitiéndonos usar el resultado de una consulta interna para definir los criterios de la consulta principal.

Son especialmente útiles cuando queremos comparar datos entre diferentes tablas o grupos de resultados dentro de la misma tabla

- Subconsultas Where: Exist & Not Exist

Las subconsultas con **EXISTS** y **NOT EXISTS** son una forma avanzada de utilizar subconsultas en SQL.

En lugar de devolver un conjunto de resultados como lo hacen otras subconsultas, **EXISTS** y **NOT EXISTS** verifican la existencia de filas que cumplen con la condición de la subconsulta.

EXISTS

La consulta principal devuelve filas si la subconsulta devuelve al menos una fila.

Supongamos que queremos seleccionar todos los empleados que tienen al menos un proyecto asignado.

Aquí, la subconsulta verifica si hay al menos un proyecto relacionado con el empleado. Si existe, la fila del empleado se devuelve.

```
SELECT "nombre", "empleado_id"
FROM "empleados"
WHERE EXISTS (
  SELECT 1
  FROM "proyectos"
  WHERE "empleados"."empleado_id" = "proyectos"."empleado_id"
);
```

Sintaxis básica de EXISTS

```
SELECT columna1, columna2
FROM tabla1
WHERE EXISTS (
  SELECT 1
  FROM tabla2
  WHERE tabla1.columnaX = tabla2.columnaY
);
```

NOT EXISTS

La consulta principal devuelve filas si la subconsulta no devuelve ninguna fila.

Si quisiéramos obtener todos los empleados que **no** tienen ningún proyecto asignado

En este caso, la consulta principal solo devolverá los empleados que no tienen proyectos asociados.

```
SELECT "nombre", "empleado_id"
FROM "empleados"
WHERE NOT EXISTS (
  SELECT 1
  FROM "proyectos"
  WHERE "empleados"."empleado_id" = "proyectos"."empleado_id"
);
```

Sintaxis básica de NOT EXISTS

```
SELECT columna1, columna2
FROM tabla1
WHERE NOT EXISTS (
    SELECT 1
    FROM tabla2
    WHERE tabla1.columnaX = tabla2.columnaY
);
```

Cuándo usar EXISTS y NOT EXISTS

- **EXISTS** es más eficiente que **IN** cuando las tablas involucradas son grandes, ya que detiene la ejecución de la subconsulta tan pronto como encuentra una coincidencia.
- **NOT EXISTS** es útil para verificar la ausencia de filas coincidentes, lo que no se puede lograr con el operador **IN**.

Diferencias entre EXISTS y otras subconsultas

A diferencia de subconsultas que devuelven un conjunto de resultados (como aquellas usadas con **IN** o **=**), **EXISTS** y **NOT EXISTS** solo verifican si la subconsulta devuelve alguna fila, lo que las hace útiles para verificar relaciones lógicas entre tablas

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- Veamos si existe algún álbum que contenga al menos una pista de tipo rock

```
SELECT "AlbumId", "Title" #Seleccionamos el álbum
FROM "Album" #De la tabla álbum
WHERE EXISTS ( #Donde exista
    SELECT 1 #Al menos una coincidencia
    FROM "Track" #De la tabla Track
    JOIN "Genre" #Unida con Género
    ON "Track"."GenreId" = "Genre"."GenreId"
    WHERE "Track"."AlbumId" = "Album"."AlbumId" #Donde los AlbumID sean iguales
    AND "Genre"."Name" = 'Rock' #Y el género sea Rock
);
```

- Obtenemos ahora los artistas que no tengan álbum asociado

```
SELECT "ArtistId", "Name" #Selecciona los artistas
FROM "Artist" #De la tabla Artist
WHERE NOT EXISTS ( #Donde no exista
    SELECT 1 #Ninguna Coincidencia
    FROM "Album" #De la Tabla Album
    WHERE "Album"."ArtistId" = "Artist"."ArtistId" #Donde coincidan los ArtistID
);
```

- Veamos ahora qué clientes han realizado alguna compra

```
SELECT "CustomerId", "FirstName", "LastName" #Seleccionamos los clientes
FROM "Customer" #De la tabla clientes
WHERE EXISTS ( #Donde exista...
    SELECT 1 #Al menos una coincidencia
    FROM "Invoice" #Con la tabla Invoice
    WHERE "Invoice"."CustomerId" = "Customer"."CustomerId" #Donde coincidan los CustomerID
);
```

- Por último, obtengamos lo contrario

```
SELECT "CustomerId", "FirstName", "LastName" #Seleccionamos los nombres de los clientes
FROM "Customer" #De la tabla Customer
WHERE NOT EXISTS ( #Los cuales no exista
    SELECT 1 #Ninguna coincidencia
    FROM "Invoice" #De la tabla Invoice
    WHERE "Invoice"."CustomerId" = "Customer"."CustomerId" #Cuyo CustomerID coincida
);
```

En conclusión...

Las subconsultas con **EXISTS** y **NOT EXISTS** son una herramienta poderosa para verificar la existencia o ausencia de filas que cumplan ciertas condiciones.

Son especialmente útiles en situaciones donde necesitamos verificar relaciones entre tablas, garantizando una mayor eficiencia en comparación con otras subconsultas.

6.2. Subconsultas con Select

Las subconsultas en el **SELECT** permiten calcular y devolver valores adicionales en las filas de la consulta principal

A diferencia de las subconsultas en **WHERE**, estas se utilizan para generar columnas adicionales basadas en otra consulta

Sintaxis básica de subconsultas en SELECT

```
SELECT columna1, columna2,
    (SELECT valor
     FROM otra_tabla
     WHERE condición) AS alias
FROM tabla;
```

Una subconsulta en **SELECT** devuelve un único valor (escalar) para cada fila de la consulta principal.

Esto es útil cuando necesitamos obtener información adicional de otra tabla o hacer cálculos basados en datos de otras consultas

Supongamos que tenemos una tabla de empleados y otra de proyectos, y queremos listar el nombre de cada empleado junto con el total de proyectos en los que está involucrado

En este ejemplo, la subconsulta en el **SELECT** cuenta el número de proyectos asociados con cada empleado y lo devuelve como una columna adicional llamada **total_proyectos**.

```
SELECT "empleado_id", "nombre",  
      (SELECT COUNT(*)  
       FROM "proyectos"  
       WHERE "proyectos"."empleado_id" = "empleados"."empleado_id") AS "total_proyectos"  
FROM "empleados";
```

Supongamos ahora que queremos obtener la cantidad de días de duración del proyecto más largo en el que cada empleado ha trabajado.

Esta subconsulta utiliza **MAX** para encontrar la duración más larga de un proyecto por empleado y mostrarla en el resultado.

```
SELECT "empleado_id", "nombre",  
      (SELECT MAX("duración")  
       FROM "proyectos"  
       WHERE "proyectos"."empleado_id" = "empleados"."empleado_id") AS "proyecto_más_largo"  
FROM "empleados";
```

Consideraciones al usar subconsultas en SELECT

- **Una subconsulta en SELECT debe devolver solo un valor** (un único resultado para cada fila de la consulta principal). Si la subconsulta devuelve más de un valor, se producirá un error.
- **Rendimiento:** Las subconsultas en el **SELECT** pueden volverse lentas si se ejecutan en grandes volúmenes de datos o tablas con muchas filas. Si es necesario, podríamos optimizarlas utilizando **JOIN** o índices adecuados

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- **Obtengamos el número total de álbumes de cada artista**

```
SELECT "Artist"."ArtistId", "Artist"."Name", #Seleccionamos los nombres de los artistas  
      (SELECT COUNT("Album"."AlbumId") #Seleccionamos el conteo de AlbumID  
       FROM "Album" #De la tabla Album  
       WHERE "Album"."ArtistId" = "Artist"."ArtistId") AS "TotalAlbums" #Cuyos ArtistID coincidan  
FROM "Artist";
```

- **Obtengamos el precio total de todas las pistas de cada álbum**

```
SELECT "Album"."AlbumId", "Album"."Title", #Seleccionamos el título de los álbumes  
      (SELECT SUM("Track"."UnitPrice") #Y la suma de los precios unitarios  
       FROM "Track" #Del la tabla Track  
       WHERE "Track"."AlbumId" = "Album"."AlbumId") AS "TotalPrice" #Cuyos AlbumID coincidan  
FROM "Album";
```

- **Obtengamos el nombre del género de cada pista**

```
SELECT "Track"."TrackId", "Track"."Name", #Seleccionamos los nombres de cada pista
      (SELECT "Genre"."Name" #Seleccionamos el género
       FROM "Genre" #De la tabla género
       WHERE "Genre"."GenreId" = "Track"."GenreId") AS "GenreName" #Cuyo GenreID coincida con la tabla Track
FROM "Track";
```

En conclusión...

Las subconsultas en el **SELECT** son útiles para calcular valores adicionales o realizar comparaciones en una sola consulta sin tener que hacer varias consultas independientes.

Nos permiten agregar información de otras tablas de manera dinámica y flexible, lo que facilita el análisis de datos y la generación de informes.

6.3. Subconsultas con From

Las subconsultas en el **FROM** permiten que el resultado de una consulta se trate como si fuera una tabla temporal, sobre la cual se pueden realizar operaciones adicionales.

Esto nos da la posibilidad de estructurar consultas más complejas y trabajar con datos intermedios generados por otra consulta dentro de la consulta principal.

Sintaxis básica de subconsultas en FROM

```
SELECT columna1, columna2
FROM (SELECT columnaX, columnaY
      FROM otra_tabla
      WHERE condición) AS alias
WHERE condición_adicional;
```

Supongamos que tenemos una tabla de ventas y queremos encontrar el promedio de ventas para los productos cuyo total de ventas sea mayor a 100.

la subconsulta en **FROM** agrupa las ventas por producto y calcula el total de ventas por producto. Luego, en la consulta principal, filtramos los productos que tengan más de 100 ventas y calculamos el promedio de las ventas.

```
SELECT "producto_id", AVG("total_ventas")
FROM (SELECT "producto_id", SUM("cantidad") AS "total_ventas"
      FROM "ventas"
      GROUP BY "producto_id") AS "ventas_agrupadas"
WHERE "total_ventas" > 100
GROUP BY "producto_id";
```

Imaginemos ahora que tenemos una tabla de empleados y otra de proyectos. Queremos obtener el total de proyectos en los que cada empleado ha participado, pero solo para aquellos empleados que han trabajado en más de 3 proyectos.

Aquí, la subconsulta en **FROM** cuenta el número de proyectos por empleado. La consulta principal selecciona solo a los empleados que han trabajado en más de 3 proyectos y los combina con la tabla de empleados.

```
SELECT "empleado_id", "nombre", "total_proyectos"
FROM (SELECT "empleado_id", COUNT("proyecto_id") AS "total_proyectos"
      FROM "proyectos"
      GROUP BY "empleado_id") AS "proyectos_agrupados"
JOIN "empleados" ON "empleados"."empleado_id" = "proyectos_agrupados"."empleado_id"
WHERE "total_proyectos" > 3;
```

Importancia de las subconsultas en FROM

- **Organización de consultas complejas:** Las subconsultas en **FROM** nos permiten descomponer consultas largas y difíciles de leer en partes más manejables. Cada subconsulta puede tratarse como una tabla separada, lo que facilita la comprensión del código.
- **Filtrado intermedio de datos:** A veces necesitamos realizar operaciones en los datos antes de ejecutar la consulta principal. Las subconsultas en **FROM** nos permiten realizar agrupaciones, sumas, filtros y otras operaciones antes de combinarlas con la lógica principal.

Consideraciones

- **Alias:** Es fundamental asignar un alias a las subconsultas en **FROM** para referirnos a ellas en la consulta principal.
- **Rendimiento:** Al igual que las subconsultas en otras partes de la consulta, si no se optimizan adecuadamente o se aplican a grandes volúmenes de datos, pueden afectar el rendimiento de la consulta.

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- **Obtengamos el total de facturas por país**

```
SELECT "BillingCountry", "TotalInvoices" #Seleccionamos el país y el total de facturas
FROM ( #De la Tabla que se genera al ...
      SELECT "Invoice"."BillingCountry", COUNT("Invoice"."InvoiceId") AS "TotalInvoices" #Seleccionar el país y el conteo de facturas
      FROM "Invoice"
      GROUP BY "Invoice"."BillingCountry" #Agrupando por país
    ) AS "CountryInvoiceCounts";
```

- **Obtengamos la suma de facturación por cliente**

SELECT "CustomerId", "FirstName", "LastName", "TotalAmount" #Seleccionamos los nombres de los clientes y el total de facturación

FROM (#De la tabla que se genera al...

SELECT "Customer"."CustomerId", "Customer"."FirstName", "Customer"."LastName",
SUM("Invoice"."Total") AS "TotalAmount" #Seleccionar los datos de los clientes junto con la suma del total de facturación

FROM "Customer" #De la tabla Customer

LEFT JOIN "Invoice" #Unida con la tabla Invoice

ON "Customer"."CustomerId" = "Invoice"."CustomerId"

GROUP BY "Customer"."CustomerId", "Customer"."FirstName", "Customer"."LastName"
#Agrupando por los datos del cliente

) AS "CustomerInvoices";

- Obtenemos el total de pistas por álbum

SELECT "AlbumId", "Title", "TotalTracks" #Seleccionamos el título de los álbumes, así como el total de canciones

FROM (#De la tabla generada al...

SELECT "Album"."AlbumId", "Album"."Title", COUNT("Track"."TrackId") AS "TotalTracks" #Seleccionar el título de los álbumes junto con el conteo de TrackId

FROM "Album" #De la tabla Album

LEFT JOIN "Track" #Unida con la tabla Track

ON "Album"."AlbumId" = "Track"."AlbumId"

GROUP BY "Album"."AlbumId", "Album"."Title" #Y agrupada por la información del álbum

) AS "AlbumTrackCounts";

En conclusión...

Las subconsultas en el **FROM** permiten que una consulta intermedia se trate como una tabla dentro de la consulta principal, proporcionando una gran flexibilidad y facilitando la creación de consultas complejas.

Nos ayudan a estructurar mejor las operaciones de filtrado, agrupación y cálculos intermedios, ofreciendo mayor control sobre los datos.

7. Estructuras de datos persistentes: Vistas

Vistas

Una **vista** es una consulta predefinida almacenada en la base de datos que actúa como una tabla virtual.

A diferencia de las tablas físicas, una vista no almacena datos, sino que genera su contenido dinámicamente al ejecutar la consulta asociada.

Se utiliza para simplificar el acceso a datos, especialmente cuando se trata de consultas complejas.

Sintaxis para crear una vista

```
CREATE VIEW nombre_vista AS
SELECT columna1, columna2
FROM tabla
WHERE condición;
```

En este ejemplo, creamos una vista llamada **vista_clientes_activos** que contiene solo los nombres y correos de los clientes activos.

```
CREATE VIEW vista_clientes_activos AS
SELECT "nombre", "email"
FROM "clientes"
WHERE "activo" = TRUE;
```

Esta consulta selecciona todos los registros de la vista previamente creada sin necesidad de repetir la lógica de selección.

```
SELECT *
FROM vista_clientes_activos;
```

Ventajas de trabajar con vistas

1 Simplificación de consultas complejas

Las vistas permiten encapsular consultas complejas en una estructura más sencilla.

Los usuarios no necesitan escribir repetidamente una consulta con varias uniones o condiciones.

Solo acceden a la vista, que ya contiene la lógica.

2 Mejora de la seguridad

Con una vista, podemos limitar el acceso a ciertas columnas o filas de una tabla.

Esto permite ofrecer una visión limitada de los datos a usuarios sin concederles acceso directo a la tabla original

3 Mantenimiento más fácil

Si las consultas utilizadas con frecuencia cambian, solo necesitamos actualizar la vista en lugar de modificar múltiples scripts en diferentes lugares.

4 Reutilización de código

Las vistas permiten reutilizar consultas complejas sin duplicar código.

Esto ahorra tiempo y esfuerzo al no tener que repetir las consultas en varios lugares.

5 Abstracción de la estructura de la base de datos

Las vistas permiten ocultar los detalles complejos de la estructura de la base de datos subyacente, presentando a los usuarios una interfaz más sencilla.

6 Optimización

Algunas bases de datos permiten la optimización de vistas mediante la materialización de vistas, que almacena temporalmente los resultados de una vista para mejorar el rendimiento de consultas recurrentes.

En conclusión...

Las vistas en SQL son una poderosa herramienta para simplificar el acceso a los datos, mejorar la seguridad, facilitar el mantenimiento y mejorar el rendimiento en algunos casos.

Trabajar con vistas permite a los desarrolladores y administradores gestionar mejor las consultas complejas y asegurar que los usuarios finales obtengan exactamente la información que necesitan

8. Estructura de datos temporales

8.1. CTEs intro

Las **CTEs** (Common Table Expressions), también conocidas como expresiones de tabla común, son una herramienta poderosa en SQL que nos permite crear tablas temporales en el contexto de una consulta.

Las CTEs son útiles cuando necesitamos realizar operaciones complejas y queremos organizar el código de manera más clara y estructurada.

Además, ofrecen una solución para evitar subconsultas repetidas o anidadas, haciéndolas más legibles y fáciles de mantener.

Sintaxis básica de las CTEs

```
WITH nombre_cte AS (  
    SELECT columnas  
    FROM tabla  
    WHERE condición  
)  
SELECT columnas  
FROM nombre_cte  
WHERE condición_adicional;
```

Ventajas de las CTEs

- **Mejoran la legibilidad del código:** Las CTEs nos permiten dividir una consulta compleja en partes más manejables. Cada CTE puede verse como un paso intermedio, lo que facilita la comprensión de la lógica.
- **Evitan duplicación de código:** Si una consulta es compleja y tiene que repetirse varias veces en diferentes partes del código, podemos definirla una sola vez como CTE y reutilizarla.
- **Temporales pero reutilizables:** Las CTEs solo existen durante la ejecución de la consulta. No crean tablas físicas, lo que las hace ligeras y eficientes.

Supongamos que queremos calcular el salario promedio de los empleados en una empresa y después usar ese dato para listar a los empleados cuyo salario sea mayor que el promedio.

```
WITH salario_promedio AS (  
    SELECT AVG("salario") AS "promedio"  
    FROM "empleados"  
)  
SELECT "nombre", "salario"  
FROM "empleados"  
WHERE "salario" > (SELECT "promedio" FROM salario_promedio);
```

CTEs con varias tablas

Las CTEs también se pueden utilizar para unir varias tablas y simplificar consultas complejas. Supongamos que queremos obtener información sobre los proyectos y los empleados asignados a ellos.

```
WITH empleados_proyectos AS (  
    SELECT "empleados"."empleado_id", "empleados"."nombre", "proyectos"."proyecto_id", "proyectos"."nombre_proyecto"  
    FROM "empleados"  
    JOIN "asignaciones" ON "empleados"."empleado_id" = "asignaciones"."empleado_id"  
    JOIN "proyectos" ON "asignaciones"."proyecto_id" = "proyectos"."proyecto_id"  
)  
SELECT *  
FROM empleados_proyectos;
```

Buenas prácticas

- **Nombres descriptivos:** Al igual que con las tablas, es importante que los nombres de las CTEs sean descriptivos para saber qué representan.
- **Reutilización:** Cuando tengamos operaciones complejas o repetidas dentro de una consulta, es ideal agruparlas en una CTE para mejorar la legibilidad

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- **Calculemos el total de facturación por cliente usando una CTE**

En este ejemplo, la CTE CustomerTotals calcula el total de facturación por cliente. Luego, en la consulta principal, se usa esta tabla temporal para mostrar a los clientes que han gastado más de 100 unidades.

```
WITH CustomerTotals AS ( --Empezamos creando nuestra CTE  
    SELECT "CustomerId", SUM("Total") AS "TotalSpent" --Seleccionamos el CustomerID y la suma del total  
    FROM "Invoice" --De la tabla Invoice  
    GROUP BY "CustomerId" --Agrupando por CustomerID  
)  
SELECT "CustomerId", "TotalSpent" --Seleccionamos las columnas creadas  
FROM CustomerTotals --De la tabla CustomerTotals  
WHERE "TotalSpent" > 100; --Con la condición de que el total sea mayor de 100
```

- **Obtengamos el total de pistas por álbum usando una CTE**

Aquí, la CTE AlbumTrackCounts cuenta el número de pistas por álbum y, en la consulta principal, se obtienen los álbumes que tienen más de 10 pistas.

```
WITH AlbumTrackCounts AS ( --Comenzamos creando nuestra CTE  
    SELECT "AlbumId", COUNT("TrackId") AS "TotalTracks" --Seleccionamos el AlbumID y el conteo de canciones de cada album  
    FROM "Track" --De la tabla Track  
    GROUP BY "AlbumId" --Y agrupamos por AlbumID  
)  
SELECT "AlbumId", "TotalTracks" --Ahora seleccionamos ambas columnas de la tabla creada en la CTE  
FROM AlbumTrackCounts  
WHERE "TotalTracks" > 10; --Pero solo aquellos álbumes con más de 10 canciones
```

- **CTE con unión de varias tablas: Artistas con más de un álbum**

En este ejemplo, la CTE ArtistAlbumCounts cuenta cuántos álbumes tiene cada artista. Luego, la consulta principal selecciona a los artistas que tienen más de un álbum.

WITH ArtistAlbumCounts AS (--Empezamos creando la CTE

```

SELECT "Artist"."ArtistId", "Artist"."Name", COUNT("Album"."AlbumId") AS "AlbumCount"
--Seleccionamos la info de los artistas, así como el conteo de AlbumID

FROM "Artist" --Desde la tabla artist

LEFT JOIN "Album" --Unida con la tabla album

ON "Artist"."ArtistId" = "Album"."ArtistId"

GROUP BY "Artist"."ArtistId", "Artist"."Name" --Agrupando por la info del artista
)

SELECT "ArtistId", "Name", "AlbumCount" --Seleccionamos las columnas de nuestra CTE

FROM ArtistAlbumCounts

WHERE "AlbumCount" > 1;--Pero sólo los registros con más de un álbum

```

En conclusión...

Las **CTEs** son una herramienta clave en SQL que nos permite crear tablas temporales dentro de una consulta.

Facilitan la organización del código, mejoran la legibilidad y evitan la duplicación de lógica compleja.

8.2. CTEs antes del SELECT

Las **CTEs (Common Table Expressions)** no solo mejoran la legibilidad y organización del código, sino que también pueden utilizarse de manera más avanzada antes del **SELECT** principal para estructurar consultas más complejas y realizar cálculos intermedios.

Esto es particularmente útil cuando queremos dividir una consulta en varios pasos, haciendo que cada etapa sea más comprensible.

Reafirmando la importancia de las CTEs

Como recordamos, una **CTE** es una especie de "tabla temporal" que solo existe durante la ejecución de la consulta. Permiten que los resultados de subconsultas o cálculos complejos se almacenen y se reutilicen dentro de la misma consulta, sin necesidad de duplicar el código o anidar múltiples subconsultas.

Se definen **antes del SELECT** y se pueden usar para realizar operaciones previas a la consulta final

Sintaxis básica de las CTEs antes del SELECT

Copiar código

```
WITH cte1 AS (  
    -- Primera CTE  
    SELECT columnas  
    FROM tabla1  
    WHERE condición  
)  
cte2 AS (  
    -- Segunda CTE que puede depender de cte1  
    SELECT columnas  
    FROM cte1  
    WHERE condición  
)  
SELECT columnas  
FROM cte2  
WHERE condición_adicional;
```

Ventajas de usar CTEs antes del SELECT

- **Modularidad:** Cuando dividimos una consulta compleja en varias etapas, podemos manejar cada parte de manera modular, verificando los resultados paso a paso.
- **Reutilización:** Las CTEs permiten que el mismo conjunto de datos sea usado varias veces dentro de la consulta, evitando cálculos redundantes.
- **Facilitan las consultas jerárquicas:** En consultas que requieren múltiples pasos de agregación o filtrado, las CTEs nos permiten ir construyendo sobre los resultados anteriores.

Imaginemos que estamos trabajando con una base de datos de ventas y queremos obtener las ventas totales por empleado, luego filtrar a aquellos empleados que superen un umbral de ventas, y finalmente listar sus nombres y el total vendido.

```
WITH ventas_totales AS (  
    -- Calculamos las ventas totales por empleado  
    SELECT "empleado_id", SUM("ventas") AS "total_ventas"  
    FROM "ventas"  
    GROUP BY "empleado_id"  
)  
empleados_destacados AS (  
    -- Filtramos empleados con más de $1000 en ventas  
    SELECT "empleado_id", "total_ventas"  
    FROM ventas_totales  
    WHERE "total_ventas" > 1000  
)  
SELECT "empleados"."nombre", "empleados_destacados"."total_ventas"  
FROM "empleados"  
INNER JOIN empleados_destacados ON "empleados"."empleado_id" = "empleados_destacados"."empleado_id";
```

Múltiples CTEs antes del SELECT

Cuando las consultas requieren múltiples transformaciones de datos, como agregaciones y cálculos basados en resultados intermedios, las **CTEs encadenadas** antes del SELECT permiten llevar a cabo estos pasos en un orden lógico.

Supongamos que tenemos una base de datos de proyectos y empleados, y queremos identificar proyectos con un gran número de empleados asignados y, posteriormente, ver el total de horas trabajadas en estos proyectos.

```
WITH proyectos_empleados AS (  
  -- Contamos cuántos empleados están asignados a cada proyecto  
  SELECT "proyecto_id", COUNT("empleado_id") AS "num_empleados"  
  FROM "asignaciones"  
  GROUP BY "proyecto_id"  
) ,  
proyectos_grandes AS (  
  -- Filtramos los proyectos con más de 5 empleados asignados  
  SELECT "proyecto_id"  
  FROM proyectos_empleados  
  WHERE "num_empleados" > 5  
) ,  
total_horas_trabajadas AS (  
  -- Calculamos el total de horas trabajadas en los proyectos grandes  
  SELECT "proyecto_id", SUM("horas_trabajadas") AS "total_horas"  
  FROM "trabajos"  
  WHERE "proyecto_id" IN (SELECT "proyecto_id" FROM proyectos_grandes)  
  GROUP BY "proyecto_id"  
)  
SELECT "proyectos"."nombre_proyecto", "total_horas_trabajadas"."total_horas"  
FROM "proyectos"  
JOIN      total_horas_trabajadas      ON      "proyectos"."proyecto_id"      =  
"total_horas_trabajadas"."proyecto_id";
```

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- **Obtengamos los clientes que se hayan gastado más de 500\$**

```
WITH TotalSales AS (--Empezamos creando nuestra CTE  
  SELECT "CustomerId", SUM("Total") AS "TotalSpent" --Seleccionamos el CustomerID y la suma de gastos totales de cada client  
  FROM "Invoice"  
  GROUP BY "CustomerId" --Agrupando por CustomerID  
)  
SELECT "CustomerId", "TotalSpent" --Seleccionamos las columnas de nuestra CTE  
FROM TotalSales  
WHERE "TotalSpent" > 500; --Pero sólo los registros de los clientes con más de 500$ de gasto
```

- Obtenemos los álbumes con más de 5 canciones

```
WITH TrackCounts AS (--Comenzamos generando nuestra CTE
    SELECT "AlbumId", COUNT("TrackId") AS "TotalTracks" --Seleccionando el identificador del álbum, así como el conteo de canciones
    FROM "Track"
    GROUP BY "AlbumId" --Agrupando por el AlbumId
)
SELECT "AlbumId", "TotalTracks" --Seleccionamos las columnas de nuestra CTE
FROM TrackCounts
WHERE "TotalTracks" > 5; --Pero solo los registros con más de 5 pistas por disco
```

- Obtenemos los nombres de los artistas que tienen más de tres álbumes.

```
WITH ArtistAlbumCounts AS (--Empezamos creando nuestra CTE de Conteo de álbumes de cada artista
    SELECT "ArtistId", COUNT("AlbumId") AS "TotalAlbums" --Seleccionamos el identificador de artista y el conteo de álbumes
    FROM "Album"
    GROUP BY "ArtistId" --Agrupando por ArtistId
),
ArtistInfo AS ( --Ahora creamos la CTE con la info de cada artista
    SELECT "ArtistId", "Name"
    FROM "Artist"
)
SELECT "ArtistInfo"."Name", "ArtistAlbumCounts"."TotalAlbums" --Seleccionamos las columnas de las CTEs
FROM ArtistAlbumCounts
INNER JOIN ArtistInfo --Unimos las CTEs
ON "ArtistAlbumCounts"."ArtistId" = "ArtistInfo"."ArtistId"
WHERE "TotalAlbums" > 3; --Seleccionamos sólo los registros con más de 3 álbumes
```

En conclusión...

Las **CTEs antes del SELECT** nos permiten realizar cálculos intermedios y organizar las consultas en pasos lógicos y fáciles de seguir.

Al dividir una consulta en múltiples CTEs, podemos estructurar nuestras consultas de forma modular y evitar duplicación de código, facilitando la legibilidad y el mantenimiento.

8.3. CTEs al comienzo de una subconsulta

En esta etapa, abordaremos cómo utilizar las **CTEs (Common Table Expressions)** al inicio de una subconsulta.

Ya hemos visto cómo las CTEs pueden ayudarnos a estructurar consultas complejas y mejorar la legibilidad del código.

Ahora, veremos cómo aplicar el mismo concepto dentro de una subconsulta, permitiendo crear cálculos y filtrados intermedios antes de que la consulta principal procese los datos.

Subconsultas

Recordemos que una **subconsulta** es una consulta anidada dentro de otra consulta principal.

Se ejecuta primero y sus resultados se utilizan en la consulta exterior.

Las subconsultas pueden estar en distintas partes de una sentencia SQL: en el **SELECT**, **FROM** o **WHERE**.

Al incorporar **CTEs al comienzo de una subconsulta**, tenemos una forma más limpia y organizada de estructurar los cálculos necesarios antes de la consulta principal.

¿Por qué usar CTEs en subconsultas?

El uso de CTEs en subconsultas nos permite:

- **Reducir la complejidad:** Dividimos consultas complejas en bloques más manejables.
- **Reutilizar resultados intermedios:** Nos permite evitar la duplicación de código cuando los mismos datos intermedios son requeridos en varias partes de la subconsulta.
- **Mejorar la legibilidad:** Hace que el código sea más fácil de seguir y mantener.

Sintaxis básica de CTEs al comienzo de una subconsulta

```
SELECT columnas
FROM (
    WITH cte AS (
        -- Subconsulta intermedia
        SELECT columnas
        FROM tabla
        WHERE condición
    )
    SELECT columnas
    FROM cte
    WHERE otra_condición
) AS subconsulta;
```

Supongamos que queremos obtener los nombres de los empleados que han trabajado más de 50 horas en proyectos específicos.

Primero, usamos una subconsulta que calcula las horas trabajadas por empleado en cada proyecto. Para organizar esto, usaremos una CTE al comienzo de la subconsulta

```
SELECT "empleados"."nombre", "horas_totales"."total_horas"
FROM "empleados"
JOIN (
    WITH horas_totales AS (
        -- Calculamos las horas totales trabajadas por cada empleado
        SELECT "empleado_id", SUM("horas_trabajadas") AS "total_horas"
        FROM "trabajos"
        GROUP BY "empleado_id"
    )
    SELECT "empleado_id", "total_horas"
    FROM horas_totales
    WHERE "total_horas" > 50
) AS horas_totales
ON "empleados"."empleado_id" = "horas_totales"."empleado_id";
```

Ventajas de usar CTEs en subconsultas

- **Claridad y organización:** En lugar de anidar subconsultas que pueden volverse difíciles de leer, usamos CTEs que permiten que cada parte de la consulta sea más clara.

- **Facilidad de mantenimiento:** Al tener las subconsultas más organizadas, es más fácil modificar, depurar y extender el código.

- **Reutilización eficiente de resultados:** Si la misma subconsulta necesita ser utilizada en varias partes de la consulta mayor, podemos definirla una vez y reutilizarla en diferentes partes del código.

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- **Obtengamos los países cuyo total de ventas sea mayor de 1000\$**

```
SELECT "BillingCountry", "TotalSales" --Seleccionamos las columnas de País y total de ventas
FROM (
    WITH CountrySales AS ( --Comenzamos a crear nuestra CTE
        SELECT "BillingCountry", SUM("Total") AS "TotalSales" --Seleccionamos el país y la suma total de ventas
        FROM "Invoice" --De la tabla Invoice
        GROUP BY "BillingCountry" --Agrupamos por País
    )
    SELECT "BillingCountry", "TotalSales" --Empezamos a crear la subconsulta
    FROM CountrySales --Seleccionamos la CTE
    WHERE "TotalSales" > 1000 --Indicamos la condición de que el total de ventas debe ser más de 1000$
) AS HighRevenueCountries;
```

- **Contar pistas de álbumes en una subconsulta para obtener aquellos con más de 3 pistas**

```
SELECT "FilteredAlbums"."AlbumId", "Title", "TotalTracks" --Seleccionemos los títulos de los álbumes y el total de pistas
FROM (
    WITH TrackCounts AS ( --Comenzamos a crear nuestra CTE
        SELECT "AlbumId", COUNT("TrackId") AS "TotalTracks" --Seleccionando el identificador de álbum y contamos el número de
        FROM "Track"
        GROUP BY "AlbumId" --Agrupamos por cada Álbum
    )
    SELECT "AlbumId", "TotalTracks" --Ahora empezamos a crear nuestra subconsulta
    FROM TrackCounts --Seleccionamos la CTE
    WHERE "TotalTracks" > 3 --Indicamos la condición de que tiene que haber más de 3 pistas
) AS "FilteredAlbums"
INNER JOIN "Album" --Por último unimos con la tabla Album
ON "Album"."AlbumId" = "FilteredAlbums"."AlbumId";
```

- **Obtengamos los clientes que gastaron más del promedio de facturación**

```
SELECT "HighSpendingCustomers"."CustomerId", "FirstName", "LastName" --Seleccionamos la información de los clientes
FROM (
    WITH CustomerTotals AS ( --Creamos nuestra CTE
        SELECT "CustomerId", SUM("Total") AS "TotalSpent" --Seleccionando el Identificador de cliente y la suma total de gasto
        FROM "Invoice" --De la tabla Invoice
        GROUP BY "CustomerId" --Agrupando por CustomerId
    )
    SELECT "CustomerId" --Ahora comenzamos la subconsulta seleccionando la columna CustomerId
    FROM CustomerTotals --De la CTE CustomerTotals
    WHERE "TotalSpent" > ( --Donde el TotalSpent debe ser mayor que la media total de facturación
        SELECT AVG("Total") FROM "Invoice"
    )
) AS "HighSpendingCustomers"
INNER JOIN "Customer" --Por último unimos con la tabla Customer
ON "Customer"."CustomerId" = "HighSpendingCustomers"."CustomerId";
```

En conclusión...

Usar **CTEs al comienzo de una subconsulta** es una excelente manera de manejar consultas complejas, dividiendo la lógica en partes más comprensibles y reutilizables.

Esto nos permite calcular datos intermedios y aplicar filtros en varias etapas de la consulta, mejorando la claridad y flexibilidad del código.

8.4. Ctes entre dos SELECT

En esta sección, abordaremos cómo podemos utilizar las **CTEs (Common Table Expressions)** entre dos sentencias **SELECT** en una consulta SQL

Esta técnica es útil cuando necesitamos reutilizar los resultados de una CTE en diferentes partes de una consulta, lo que nos permite mantener un código más limpio y organizado.

¿Cuándo se usan CTEs entre dos SELECT?

Cuando tenemos consultas donde los resultados de un **SELECT** necesitan ser utilizados en otro **SELECT**, la implementación de **CTEs** nos permite reutilizar los mismos datos sin duplicar código.

Esto es especialmente útil en casos donde los mismos cálculos o filtrados intermedios son necesarios en varias partes de la consulta.

Sintaxis básica de CTEs entre dos SELECT

```
WITH cte_nombre AS (  
  -- Primer SELECT: Definimos los datos intermedios  
  SELECT columnas  
  FROM tabla  
  WHERE condición  
)  
-- Segundo SELECT: Utilizamos los resultados de la CTE  
SELECT columnas  
FROM cte_nombre  
WHERE otra_condición;
```

Supongamos que queremos obtener los datos de los empleados que han trabajado en proyectos y han acumulado más de 50 horas, pero queremos reutilizar el cálculo de las horas trabajadas en varias consultas. En este caso, las **CTEs** nos permitirán hacer esto de forma más eficiente.

```
WITH horas_trabajadas AS (  
  -- Primer SELECT: Calculamos las horas totales trabajadas por cada empleado  
  SELECT "empleado_id", SUM("horas_trabajadas") AS "total_horas"  
  FROM "trabajos"  
  GROUP BY "empleado_id"  
)  
-- Segundo SELECT: Utilizamos los resultados para filtrar empleados con más de 50 horas trabajadas  
SELECT "empleado_id", "total_horas"  
FROM horas_trabajadas  
WHERE "total_horas" > 50;
```

Ventajas de usar CTEs entre dos SELECT

- **Reutilización de resultados:** En lugar de repetir los cálculos en diferentes partes de una consulta, podemos definirlos una vez y usarlos en varios **SELECT**.
- **Simplicidad:** Ayuda a simplificar consultas complejas, permitiendo dividir las operaciones en diferentes bloques lógicos.
- **Flexibilidad:** Nos permite realizar cálculos intermedios que pueden ser referenciados en varias partes de la consulta, evitando redundancias.

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- **Calculemos las ventas por país que superan un umbral**

```
WITH CountrySales AS (-- Empezamos creando la CTE
-- Empezamos calculando el total de ventas por país
SELECT "BillingCountry", SUM("Total") AS "TotalSales" --Seleccionamos el país y la suma de las ventas
FROM "Invoice"
GROUP BY "BillingCountry" --Agrupamos por país
)
-- Seguimos con el segundo SELECT: seleccionar países con ventas superiores a 1000
SELECT "BillingCountry", "TotalSales" --Seleccionamos los países y el total de ventas
FROM CountrySales
WHERE "TotalSales" > 1000; --Añadimos la condición de que las ventas deben superar los 1000$
```

- **Obtengamos los Álbumes con más de 10 pistas**

```
WITH AlbumTrackCounts AS (--Comenzamos creando la CTE
-- Empezamos con el primer SELECT: contar pistas por álbum
SELECT "AlbumId", COUNT("TrackId") AS "TrackCount" -- Seleccionamos el identificador de Álbum y el conteo de pistas
FROM "Track"
GROUP BY "AlbumId" --Agrupamos por el identificador de Album
)
-- Seguimos con el segundo SELECT: seleccionar álbumes con más de 10 pistas
SELECT "AlbumId", "TrackCount" --Seleccionamos el identificador de Álbum y el conteo de pistas
FROM AlbumTrackCounts
WHERE "TrackCount" > 10; --Añadimos la condición de que los álbumes deben tener más de 10 pistas
```

- **Obtengamos los clientes con más de una factura**

WITH CustomerInvoices AS (--Creamos la CTE

-- Comenzamos con el primer SELECT: contar facturas por cliente

SELECT "CustomerId", COUNT("InvoiceId") AS "InvoiceCount" -- Seleccionamos el identificador de cada cliente junto con el conteo de las facturas

FROM "Invoice"

GROUP BY "CustomerId" --Agrupamos por el identificador del cliente

)

-- Seguimos con el segundo SELECT: seleccionar clientes con más de una factura

SELECT "CustomerId" -- Seleccionamos el identificador de cliente

FROM CustomerInvoices -- De la CTE

WHERE "InvoiceCount" > 1; --Donde el conteo de Facturas sea mayor que 1

En conclusión...

El uso de **CTEs entre dos SELECT** es una estrategia poderosa para mantener el código SQL limpio y eficiente, permitiendo realizar cálculos intermedios que pueden ser reutilizados en varias partes de una consulta.

Esto no solo simplifica la escritura del código, sino que también mejora su legibilidad y mantenibilidad, especialmente en consultas complejas o que involucran múltiples pasos.

8.5. Tablas temporales

En esta sección, nos centraremos en las **tablas temporales**, que son una herramienta poderosa en SQL cuando necesitamos almacenar datos de manera temporal para un procesamiento o análisis intermedio dentro de una sesión.

A diferencia de las **CTEs**, que son temporales dentro de una consulta específica, las **tablas temporales** permiten almacenar datos temporalmente durante una sesión completa de trabajo, y se eliminan cuando la sesión o conexión se cierra.

¿Qué es una tabla temporal?

Una **tabla temporal** es una tabla que existe de manera temporal en la base de datos y solo es visible para la sesión de usuario que la ha creado. Los datos almacenados en esta tabla pueden ser manipulados y consultados como cualquier otra tabla regular, pero se eliminan automáticamente al finalizar la sesión.

Sintaxis de creación de tablas temporales

```
CREATE TEMPORARY TABLE nombre_tabla_temporal (  
    columna1 tipo_dato,  
    columna2 tipo_dato,  
    ...  
);
```

Supongamos que quisiéramos crear una tabla temporal para almacenar información de ventas temporalmente, podríamos escribir

```
CREATE TEMPORARY TABLE ventas_temporales (  
    "id_venta" INT,  
    "monto" DECIMAL(10, 2),  
    "fecha_venta" DATE  
);
```

Ventajas de las tablas temporales

- **Almacenamiento temporal de datos:** Son útiles cuando necesitamos realizar cálculos intermedios o almacenar datos temporalmente para otras consultas posteriores.
- **Sesión aislada:** Los datos almacenados en una tabla temporal solo son visibles dentro de la sesión que la ha creado, lo que asegura que otros usuarios no puedan acceder a ellos.

- **Automáticamente eliminadas:** No tenemos que preocuparnos por eliminar las tablas temporales manualmente; se eliminan automáticamente al cerrar la sesión.
- **Flexibilidad en análisis complejo:** Nos permiten dividir consultas complejas en pasos más sencillos, guardando resultados intermedios que luego pueden ser reutilizados en múltiples consultas.

Imaginemos que estamos analizando los datos de ventas de una empresa y queremos realizar varias consultas sobre las ventas en un trimestre específico.

Podemos almacenar esos datos filtrados en una tabla temporal y luego realizar diferentes análisis sobre ellos

```
-- Crear una tabla temporal con las ventas del primer trimestre
CREATE TEMPORARY TABLE ventas_q1 AS
SELECT "id_venta", "monto", "fecha_venta"
FROM "ventas"
WHERE "fecha_venta" BETWEEN '2024-01-01' AND '2024-03-31';

-- Consultar el monto total de las ventas del primer trimestre
SELECT SUM("monto") AS "total_ventas_q1"
FROM ventas_q1;

-- Consultar el número de ventas en el primer trimestre
SELECT COUNT(*) AS "num_ventas_q1"
FROM ventas_q1;
```

Tipos de tablas temporales

Existen dos tipos principales de tablas temporales:

- **Tablas temporales locales:** Solo existen dentro de la sesión de trabajo del usuario que las creó. Se eliminan automáticamente cuando la sesión finaliza.
- **Tablas temporales globales:** Existen hasta que todas las sesiones que las están utilizando se cierren. Otros usuarios pueden acceder a ellas si la tabla fue definida como global, pero los datos se gestionan por sesión.

Orden de ejecución

1 Se crean y se llenan las tablas temporales en el **FROM** o con una sentencia **CREATE TEMPORARY TABLE**.

2 Luego, se pueden realizar selecciones o manipulaciones de los datos temporales con **SELECT**, **UPDATE**, **DELETE**, etc.

Veamos ahora unos ejemplos para nuestra BBDD Chinook

- Empecemos almacenando y consultando las ventas totales por cliente

Primero creamos la tabla temporal

```
CREATE TEMPORARY TABLE TempCustomerSales AS --Creamos la tabla temporal
SELECT "CustomerId", SUM("Total") AS "TotalSpent" -- Seleccionamos el identificador de cliente y la suma de dinero gastado
FROM "Invoice"
GROUP BY "CustomerId"; --Agrupamos por identificador de cliente
```

Creamos la consulta

```
SELECT "CustomerId", "TotalSpent"
FROM TempCustomerSales
WHERE "TotalSpent" > 1000;
```

- Almacenemos pistas de los álbumes de un artista y consultemos las pistas largas

Creamos la tabla temporal

```
CREATE TEMPORARY TABLE TempArtistTracks AS
SELECT "Track"."TrackId", "Track"."Name", "Track"."Milliseconds"
FROM "Track"
JOIN "Album" ON "Track"."AlbumId" = "Album"."AlbumId"
JOIN "Artist" ON "Album"."ArtistId" = "Artist"."ArtistId"
WHERE "Artist"."Name" = 'AC/DC';
```

Seguimos con la consulta

```
SELECT "TrackId", "Name", "Milliseconds"
FROM TempArtistTracks
WHERE "Milliseconds" > 300000;
```

Almacenamos los álbumes con más de 10 pistas y consultamos los álbumes más populares

Creamos la tabla temporal

```
CREATE TEMPORARY TABLE TempAlbumsWithManyTracks AS
SELECT "Album"."AlbumId", "Album"."Title", COUNT("Track"."TrackId") AS "TrackCount"
FROM "Album"
JOIN "Track" ON "Album"."AlbumId" = "Track"."AlbumId"
GROUP BY "Album"."AlbumId"
HAVING COUNT("Track"."TrackId") > 10;
```

Generamos la consulta

```
SELECT "AlbumId", "Title", "TrackCount"
FROM TempAlbumsWithManyTracks;
```

En conclusión...

Las **tablas temporales** son una herramienta esencial para almacenar datos de forma temporal durante una sesión de trabajo.

Nos permiten hacer cálculos intermedios, facilitar análisis complejos y mejorar la organización del código SQL.

Son eliminadas automáticamente al final de la sesión, lo que las convierte en una opción eficiente y flexible para el análisis temporal de datos.